西南交通大学
Southwest Jiaotong University

《网络空间安全技术》课程设计报告

设计题目：Web 登录系统安全加固与审计日志设计

学　　号：2023112621

姓　　名：程泽凯

班　　级：计算机类 2023 级

指导教师：郑丹阳　　老师

2026 年 6 月 10 日

目录

一、 课程设计题目 1

二、 项目背景 1

三、 实验环境 1

四、 需求分析 2

4.1 功能需求 2

4.2 安全需求 3

4.3 异常规则需求与个性化阈值设计 3

五、 技术方案 4

5.1 系统总体架构 4

5.2 密码哈希方案 5

5.3 数据库设计 6

5.4 登录失败限制、二次验证与异常检测策略 6

六、 系统总体流程图 7

七、 核心模块流程图 7

7.1 用户登录认证流程 8

7.2 登录失败限制流程 8

八、 测试与处置流程图 9

8.1 异常登录审计流程 9

8.2 自动化测试与异常处置流程 10

九、 系统实现 11

9.1 登录认证核心 11

9.2 异常检测实现 12

9.3 Web 页面、二次验证与异常提醒 12

十、 实验数据 13

10.1 测试账号（本人创建） 13

10.2 模拟异常登录记录（本人设计） 14

10.3 审计真实输出 15

十一、 测试用例 16

十二、 运行结果 17

十三、 问题与改进 18

13.1 遇到的问题及解决 18

13.2 改进方向 18

13.3 答辩可现场验证的参数 19

十四、 总结 19

十五、 附录：代码、配置、日志、截图 19

15.1 项目代码索引 19

15.2 关键配置 20

参考文献 20

一、 课程设计题目

题目：Web 登录系统安全加固与审计日志设计。

本课程设计由本人程泽凯（学号 2023112621）独立完成。围绕一个本地运行的小型 Web 登录系统，本人完成了需求分析、技术选型、数据库与安全规则设计、Python 编码、模拟数据构造、测试用例编写、流程图绘制、实验验证以及 LaTeX 报告排版。项目重点不是简单实现登录页面，而是在可运行的登录流程中加入口令安全存储、登录二次验证模拟、失败限制、账号锁定、审计留痕和异常识别机制，并通过本人编写的测试脚本验证设计效果。

合规声明：本课程设计全部在本机（Windows 11 家庭中文版 + Python 3.13）离线环境与本地浏览器中完成。数据库、登录日志、测试账号与异常登录记录均为本人自行生成的模拟数据，**未对任何真实第三方系统、公网服务器或校园网设备进行扫描、攻击或测试，未造成任何影响。**

二、 项目背景

登录系统是 Web 应用的安全入口，也是攻击者的首要目标。一旦登录环节被攻破，攻击者即可获得合法身份并进一步访问受保护资源。常见的登录安全威胁包括：

- **口令明文或弱哈希存储：**数据库一旦泄露，口令直接暴露，并可能被用于撞库攻击；
- **暴力破解与撞库：**攻击者借助自动化脚本对账号进行高频口令尝试；
- **异常登录缺乏感知：**异地登录、非工作时间登录、高频请求等可疑行为得不到及时识别；
- **缺乏审计能力：**登录行为无日志记录，安全事件发生后无法追溯定位。

针对上述问题，OWASP 在其认证安全指南中明确建议对口令进行加盐慢哈希存储、对失败登录进行限制并记录审计日志 [4]；NIST SP 800-63B 数字身份指南同样强调了口令存储与认证生命周期管理的安全要求 [1]。本课程设计正是围绕这些安全实践，构建一个具备**安全加固与审计能力**的小型登录系统，做到“有场景、有数据、有流程、有规则、有实现、有测试、有结果、有分析”。

三、 实验环境

本系统的开发与测试环境如表 1 所示。选型上采用 Python Flask + SQLite，原因在于二者轻量、零外部服务依赖，便于本地一键运行与答辩现场演示；同时不引入 ORM，所有 SQL 透明可审计，便于讲解与安全分析。

表 **1**　实验环境配置

类别	配置
操作系统	Windows 11 家庭中文版（10.0.26200）
开发语言	Python 3.13.5
Web 框架	Flask 3.1.0 / Werkzeug 3.1.3
数据库	SQLite 3（Python 标准库 `sqlite3`）
口令哈希	PBKDF2-SHA256，26 万次迭代，自带随机盐
运行与测试	本地浏览器（127.0.0.1:5000）+ 自动化测试脚本

图 1 为本人在本机 PowerShell 中执行 `python --version` 与 `pip show flask` 后截取的实验环境信息。

```
(base) PS D:\大三下\网络空间安全技术\课设> python --version
Python 3.13.5
(base) PS D:\大三下\网络空间安全技术\课设> pip show flask
Name: Flask
Version: 3.1.0
Summary: A simple framework for building complex web applications.
Home-page:
Author:
Author-email:
License:
Location: D:\Anaconda\Lib\site-packages
Requires: blinker, click, itsdangerous, Jinja2, Werkzeug
Required-by: flask-cors
```

图 **1**　本人本机 **Python** 与 **Flask** 实验环境

四、 需求分析

4.1 功能需求

任务书要求“必须实现的功能”共 7 项，本系统逐条实现并定位到对应代码模块，如表 2 所示。

表 **2**　必做功能与实现位置对照

序号	必须实现的功能	实现位置
1	用户注册	`app.py` / `security.register_user`
2	用户登录	`app.py` / `security.authenticate`
3	密码哈希存储	`security.hash_password`
4	登录失败计数	`security.count_recent_failures`
5	账号临时锁定	`security` 锁定逻辑 + `locked_until`
6	登录日志记录	`db.add_login_log` + `login_logs` 表
7	异常登录提醒	`audit.detect_realtime`

4.2 安全需求

- 口令绝不明文存储，采用加盐慢哈希；
- 用户名不存在与口令错误返回统一提示，防止用户名枚举；
- 所有数据库操作使用参数化查询（占位符 `?`），防止 SQL 注入；
- 失败登录达阈值后临时锁定账号，抵抗暴力破解；
- 登录页加入固定码 `12345` 的二次验证模拟，只有用户名、口令与验证码均通过后才进入认证成功流程；
- 审计可重复执行并输出结构化结果，便于事后追溯。

4.3 异常规则需求与个性化阈值设计

任务书给出四类异常的规则示例，本人结合自身上机作息与实验场景，将其细化为可量化、可现场修改的阈值，如表 3 所示。所有阈值集中存放于 `config.py`，答辩时可现场修改任一数值并重新运行验证。

表 **3**　四类异常的个性化判断规则（设计者：程泽凯）

异常类型	任务书规则示例	本人设计的判断规则
暴力尝试	5 分钟内失败超过 5 次	5 分钟窗口内失败 ≥ 5 次 $\rightarrow$ 锁定 10 分钟
异地登录	同账号短时不同 IP	10 分钟内同账号出现 ≥ 2 个不同 IP
非工作时间	晚 23:00 后登录	23:00–06:00 之间成功登录
高频登录	1 分钟内超过阈值	60 秒内请求 ≥ 10 次

五、 技术方案

5.1 系统总体架构

系统采用分层设计，由配置层、数据访问层、安全核心层、审计层与 Web 表现层构成。各模块职责单一、低耦合，所有安全阈值与模拟二次验证码集中于配置层。整体调用关系如下：浏览器经 HTTP 访问 Flask 路由层（`app.py`），路由层先校验登录页输入的二次验证模拟码 `12345`，再调用安全核心（`security.py`）完成认证，调用审计模块（`audit.py`）完成异常检测，二者均通过数据访问层（`db.py`）读写 SQLite 数据库。

图 2 展示了本人本地项目目录。代码按配置、数据访问、安全认证、异常审计、页面模板与自动化测试拆分，便于逐模块验证。



图 **2**　本人本地项目目录结构

5.2 密码哈希方案

系统采用 Werkzeug 提供的 `pbkdf2:sha256`（26 万次迭代）对口令进行加盐慢哈希存储，符合 PKCS #5 / RFC 8018 对基于口令的密钥派生函数的规范 [2]。其安全意义在于三点：（1）**加盐**——每个口令使用独立随机盐，相同口令的哈希值各不相同，可抵抗彩虹表攻击；（2）**慢哈希**——26 万次迭代显著抬高离线爆破成本；（3）**恒定时间比较**——校验时使用恒定时间比较，防止时序侧信道泄露。核心实现如代码清单 1 所示。

```python
from werkzeug.security import generate_password_hash, check_password_hash

# Store: salted slow hash, never plaintext
def hash_password(plain):
    return generate_password_hash(plain, method="pbkdf2:sha256:260000")

```

```
# Verify: constant-time comparison
def verify_password(stored_hash, plain):
    return check_password_hash(stored_hash, plain)
```

Listing 1: 口令哈希与校验（security.py 节选）

5.3 数据库设计

数据库包含三张表:用户表 users、登录日志表 login_logs 与审计告警表 audit_alerts，并为日志的高频查询字段建立联合索引，如代码清单 2 所示。其中 login_logs.reason 取值为 ok / wrong_password / user_not_found / locked，是后续审计判断的关键字段。

```
CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  created_at TEXT NOT NULL,
  failed_count INTEGER NOT NULL DEFAULT 0,
  locked_until TEXT,       -- lock deadline, NULL = not locked
  last_login_at TEXT
);
CREATE TABLE login_logs (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT NOT NULL, ip TEXT NOT NULL, user_agent TEXT,
  success INTEGER NOT NULL, -- 1 success / 0 failure
  reason TEXT NOT NULL, created_at TEXT NOT NULL
);
CREATE TABLE audit_alerts (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT NOT NULL, alert_type TEXT NOT NULL,
  detail TEXT NOT NULL, created_at TEXT NOT NULL
);
CREATE INDEX idx_logs_user_time ON login_logs(username, created_at);
```

Listing 2: 数据库表结构（db.py 节选）

5.4 登录失败限制、二次验证与异常检测策略

失败计数采用**基于日志的时间窗口统计**而非简单计数器累加，因此“5 分钟内失败 5 次”这一时间语义是精确的。当窗口内失败次数达到阈值，即写入锁定截止时间并生成暴力尝试告警。为满足登录加固中的验证码或二次验证要求，本人另外在登录页加入固定码 12345 的二次验证模拟：验证码错误时，系统不进入后续口令校验与会话写入，

直接提示“二次验证失败”；验证码正确时，才继续调用 `security.authenticate()`。异常检测分为两个阶段：登录成功瞬间运行轻量的**实时检测**（异地 / 非工作时间 / 高频），命中即弹出提醒；离线运行**全量审计**，按时间回放全部日志、对四类规则逐条匹配并去重，输出结构化报告。

六、 系统总体流程图

为使系统结构与本人编写的代码能够相互对应，本节给出系统总体处理流程。浏览器请求首先进入 `app.py` 路由层，再根据用户选择进入注册、登录或安全审计功能；登录认证与异常审计的细化步骤将在后续章节展开。

图 3 给出系统的总体处理流程，对应 `app.py` 的路由分发。

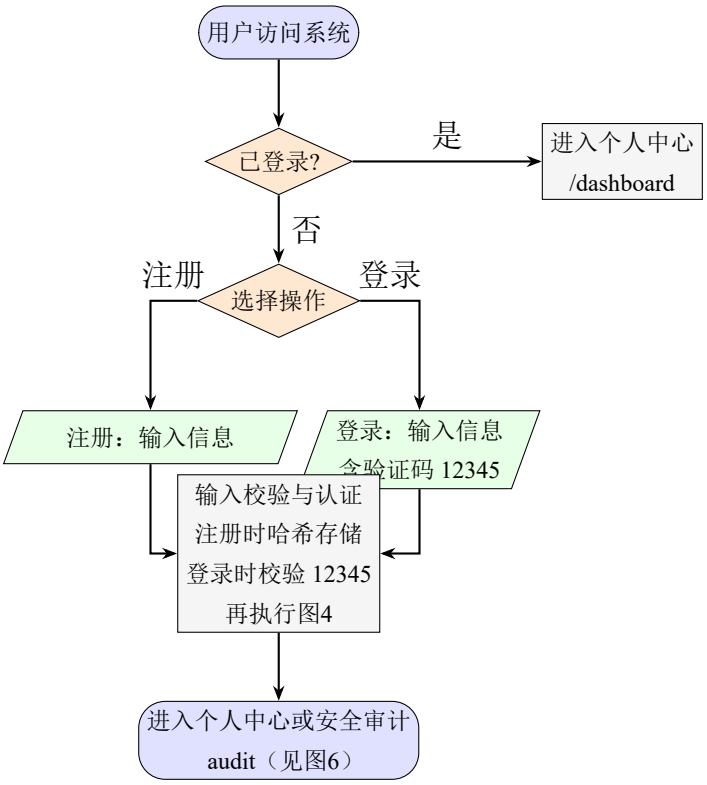


图 **3**　系统总体流程图

七、 核心模块流程图

本节给出两个关键安全模块的流程：用户登录认证与登录失败限制。两张流程图均由本人根据实际代码分支绘制，可用于答辩时逐项说明“流程图、代码与安全规则”的对应关系。

7.1 用户登录认证流程

图 4 对应 `security.authenticate()` 函数，完整体现“查询用户 → 判断锁定 → 判断用户是否存在 → 校验口令 → 成功后实时异常检测”的判断分支与异常路径。

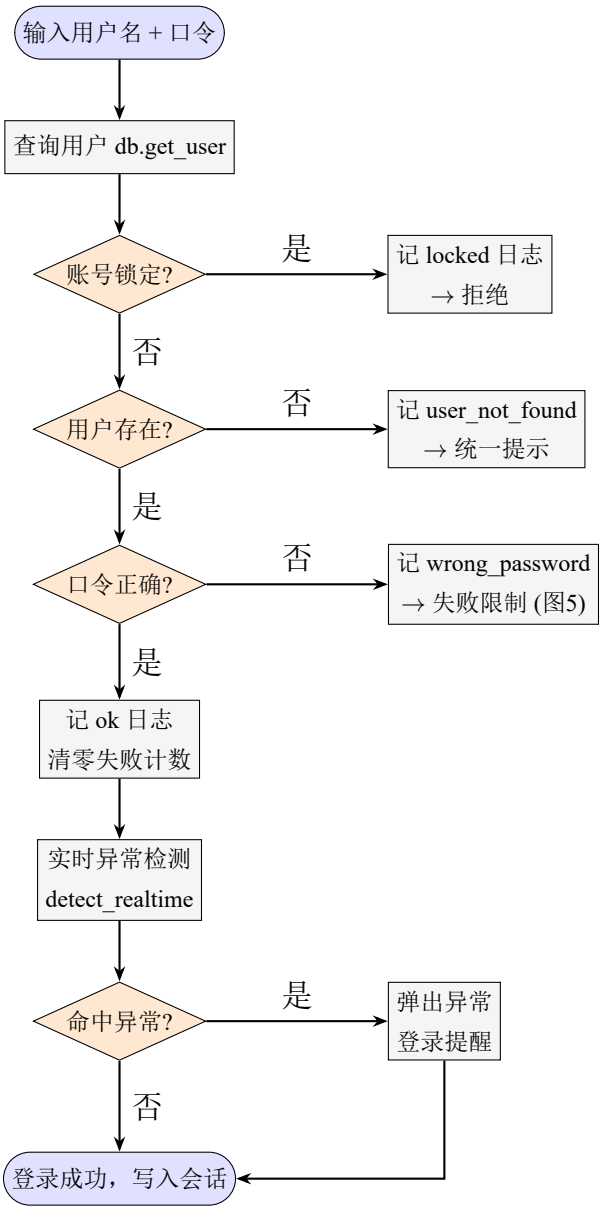


图 **4**　用户登录认证流程图

7.2 登录失败限制流程

图 5 对应口令错误分支与 `count_recent_failures()`：失败计数加一、判断阈值、锁定账号、记录告警。

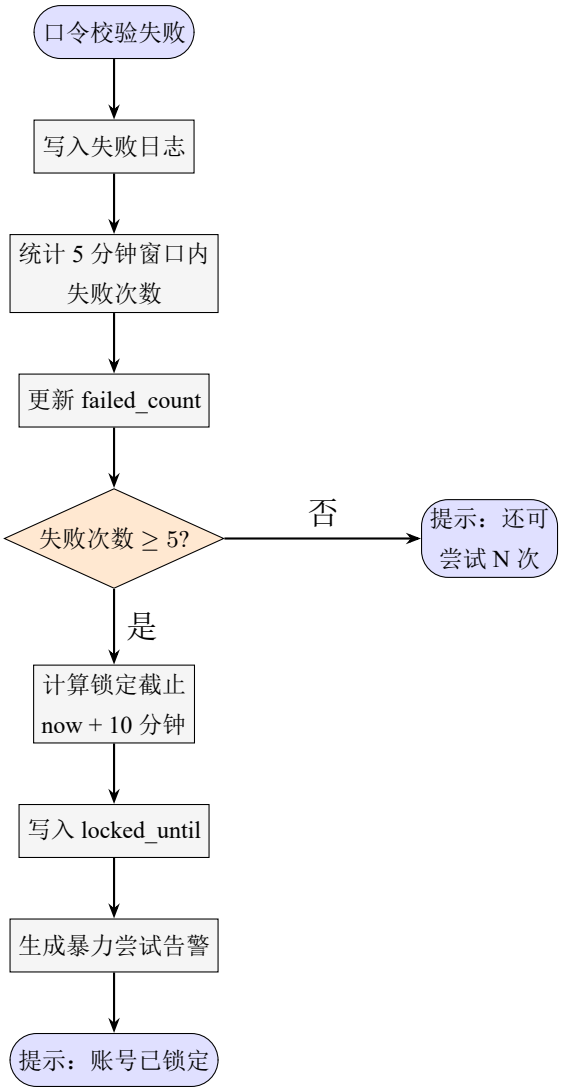


图 **5**　登录失败限制流程图

八、 测试与处置流程图

本节给出异常登录审计流程与测试处置流程。前者对应本人实现的 `audit.py`，后者对应本人编写的 `run_tests.py` 与异常处置策略。通过将“检测”和“处置”连成闭环，可以更清楚地体现系统的实际安全用途。

8.1 异常登录审计流程

图 6 对应 `audit.run_full_audit()`：清空旧告警、按时间遍历日志、对四类规则匹配、汇总输出审计结果。

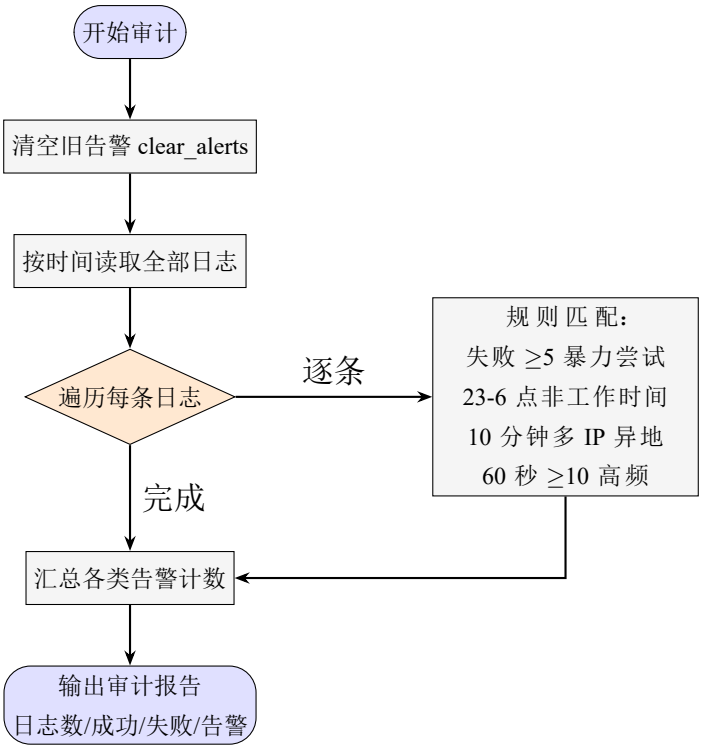


图 **6**　异常登录审计流程图

8.2 自动化测试与异常处置流程

图 7 对应 `run_tests.py` 与系统的异常处置思路。每个测试用例执行前先重建数据库，避免不同用例互相污染；若用例未通过，则返回检查规则、阈值与实现。告警产生后，根据不同异常类型采取锁定、提醒、人工复核或限流等处置方式。

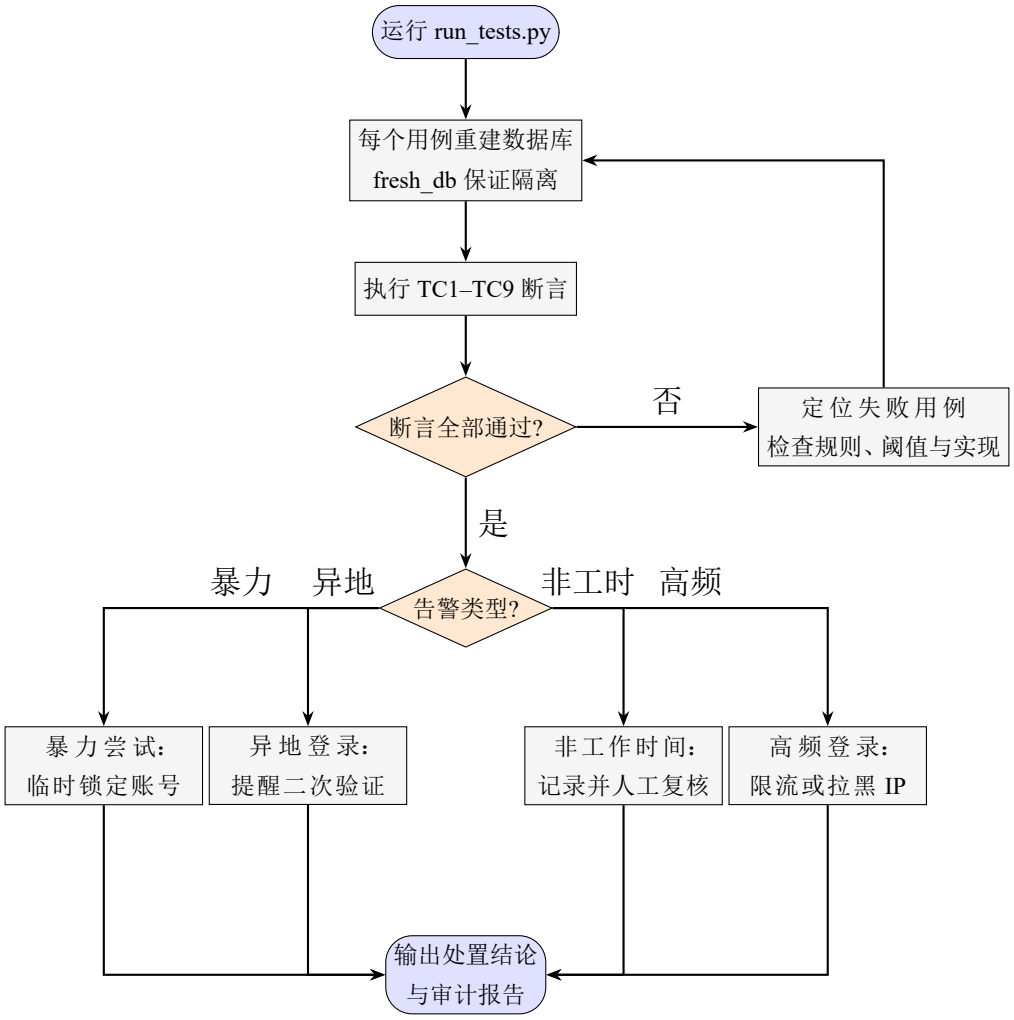


图 7　自动化测试与异常处置流程图

九、 系统实现

9.1 登录认证核心

登录认证严格按图 4 的四个分支实现：账号锁定则直接拒绝；用户不存在返回统一提示防枚举；口令错误进入失败限制流程；认证成功后清零计数并运行实时异常检测。核心逻辑如代码清单 3 所示。

```
def authenticate(username, password, ip, user_agent):
    with db.get_conn() as conn:
        user = db.get_user(conn, username)
        # branch 1: account locked -> reject directly
        if user is not None and is_locked(user):
            db.add_login_log(conn, username, ip, user_agent, False, "locked", ts)
            return {"success": False, "reason": "locked", ...}
        # branch 2: user not found -> unified message (anti-enumeration)
        if user is None:
```

```
            db.add_login_log(conn, username, ip, user_agent, False, "user_not_found", ts)
            return {"success": False, "reason": "user_not_found", ...}
        # branch 3: wrong password -> count + maybe lock
        if not verify_password(user["password_hash"], password):
            db.add_login_log(conn, username, ip, user_agent, False, "wrong_password", ts)
            failures = count_recent_failures(conn, username)
            if failures >= MAX_FAILED_ATTEMPTS:
                db.set_lock(conn, username, lock_until) # lock 10 min
                db.add_alert(conn, username, "brute_force", ...)
            return {"success": False, "reason": "wrong_password", ...}
        # branch 4: success -> reset + realtime anomaly detection
        db.add_login_log(conn, username, ip, user_agent, True, "ok", ts)
        db.reset_user_security(conn, username, ts)
        alerts = audit.detect_realtime(conn, username, ip, now())
    return {"success": True, "reason": "ok", "alerts": alerts}
```

Listing 3: 登录认证主流程（security.authenticate 节选）

9.2 异常检测实现

非工作时间判断需处理跨午夜区间（23:00 至次日 06:00），异地登录通过统计窗口内不同 IP 数实现，高频登录通过秒级窗口计数实现，如代码清单 4 所示。

```
def is_off_hours(dt):          # rule 3: cross-midnight interval
    h = dt.hour
    start, end = 23, 6
    return h >= start or h < end

def distinct_ips_in_window(conn, username, end_dt, minutes): # rule 2
    start = (end_dt - timedelta(minutes=minutes)).isoformat(sep=" ")
    cur = conn.execute("SELECT DISTINCT ip FROM login_logs "
        "WHERE username=? AND created_at>=? AND created_at<=?",
        (username, start, end_dt.isoformat(sep=" ")))
    return [r["ip"] for r in cur.fetchall()]
```

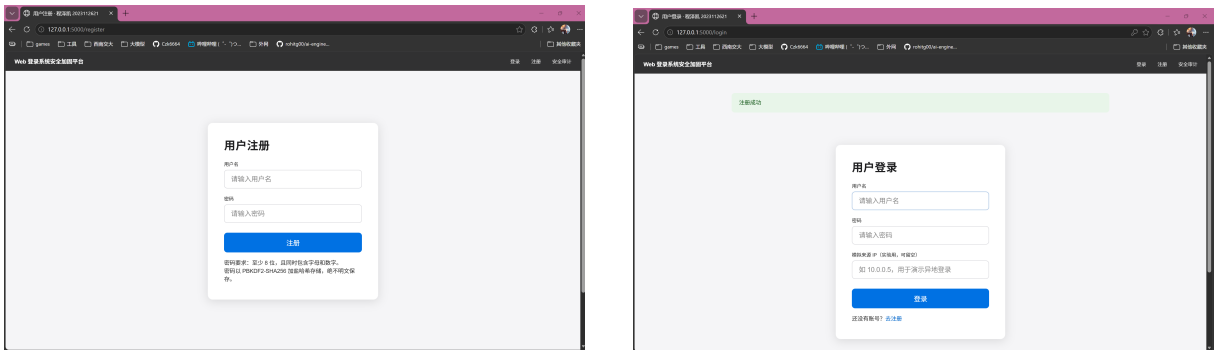
Listing 4: 异常规则判断（audit.py 节选）

9.3 Web 页面、二次验证与异常提醒

为便于在答辩现场演示，本人实现了注册页、登录页、个人中心与安全审计页。登录页除用户名和口令外，还加入“二次验证模拟码”输入框，实验固定值为 12345；若输入其他值，app.py 会在调用认证函数前直接拒绝，防止仅凭口令进入系统。登录页还额外提供“模拟来源 IP”输入框，仅用于本地实验中构造异地登录记录。图 8 展示了注册页和登录页，图 9 展示了同一账号短时间使用多个模拟 IP 登录后触发的异常提醒。

```python
otp_code = request.form.get("otp_code", "").strip()
if otp_code != config.SIMULATED_OTP_CODE:
    flash("二次验证失败：请输入本地实验验证码 12345", "danger")
    return render_template("login.html")
```

Listing 5: 二次验证模拟码校验（app.py 节选）



(a) 用户注册页面　　**(b)** 用户登录页面

图 **8**　本人实现的 **Web** 注册与登录页面

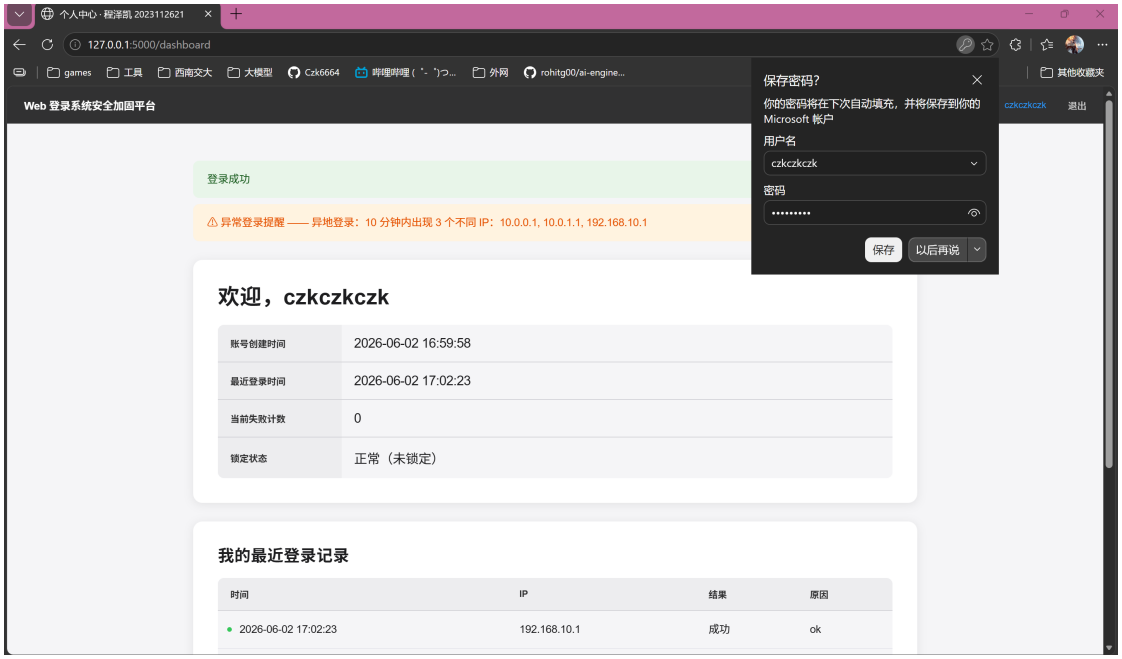


图 **9**　本人模拟多个来源 **IP** 后触发异地登录提醒

十、 实验数据

10.1 测试账号（本人创建）

本人在 `seed_data.py` 中创建 4 个测试账号，口令均经加盐哈希存储，如表 4 所示。

表 **4**　本人创建的测试账号

用户名	用途	口令（已加盐哈希存储）
czk2023	主账号（程泽凯本人）	Czk@2023112621
alice99	普通用户	Alice#pass01
bob_dev	普通用户	BobDev2024!
victim7	演示被暴力破解的受害账号	Victim#0001

口令哈希的真实存储样例（库中无任何明文）：

```
czk2023 -> pbkdf2:sha256:260000$W10OrlyPNRYsjpci$8bfdef18688201d9f...
alice99 -> pbkdf2:sha256:260000$77QdBCxQwoKr8Zoy$01689276b56190d39...
```

图 10 为本人从本地 SQLite 数据库查询得到的口令哈希截图。可以看到，数据库只保存带随机盐的 PBKDF2-SHA256 哈希，不保存明文口令。

```
(base) PS D:\大三下\网络空间安全技术\课设> python -c "import sqlite3; c=sqlite3.connect('da
ta/secure_login.db'); [print(u, '->', h[:75] + '...') for u,h in c.execute('select username
,password_hash from users')]"
czk2023 -> pbkdf2:sha256:260000$Hv1w5F8fTPFPml0C$7ec3d0f0e5c6074d8b020a9309548d71e7342...
alice99 -> pbkdf2:sha256:260000$ooDJ5Bl935fvxq67$07b86747eb4df64302bdceea8bb583f13b3ac...
bob_dev -> pbkdf2:sha256:260000$Zmj4Tr2uaz8AVTPJ$1939cbf8df1baf19e11df3d820e336d924310...
victim7 -> pbkdf2:sha256:260000$3oRDuBJlp10vv1CE$1729c037ae8375b0a095ac338715281b167e8...
czkczkczk -> pbkdf2:sha256:260000$ihh49wHa8xNMwiUQ$1dd935103cc12a9004b1f9d21ef26d037b6d7...
```

图 **10**　本人本地数据库中的口令哈希存储结果

10.2 模拟异常登录记录（本人设计）

为覆盖四类异常并保留一组正常对照，本人设计了如表 5 所示的模拟数据。

表 **5**　本人设计的模拟登录场景

场景	账号	设计的数据
A 暴力尝试	victim7	3 分钟内连续失败 6 次
B 异地登录	alice99	4 分钟内来自 10.0.0.5 / 172.16.8.9 / 203.0.113.7
C 非工作时间	bob_dev	凌晨 02:30 成功登录
D 高频登录	czk2023	40 秒内尝试 12 次
E 正常对照	czk2023	白天 14:00 单 IP 低频登录（不应告警）

10.3 审计真实输出

对上述种子数据运行全量审计，本机真实输出为：日志总数 23 条，其中成功登录 6 条、失败登录 17 条，共生成 10 条告警，分类统计如表 6 所示。

表 **6**　审计告警分类统计（本机真实结果）

异常类型	命中次数
暴力尝试	4
异地登录	2
非工作时间登录	1
高频登录	3
合计	10

部分告警明细（真实截取）：

```
[非工作时间登录] bob_dev 于 2026-05-26 02:30:00 在非工作时段成功登录
[暴力尝试]     victim7 截至 2026-05-26 19:10:56, 5 分钟内失败 5 次
[异地登录]     alice99 10 分钟内出现 3 个 IP: 10.0.0.5, 172.16.8.9, 203.0.113.7
[高频登录]     czk2023 60 秒内请求 12 次
```

图 11 与图 12 为本人本地安全审计页面截图，分别展示告警明细与最近登录日志。页面中可以看到本人实际触发的异地登录、暴力尝试以及多条登录失败记录。

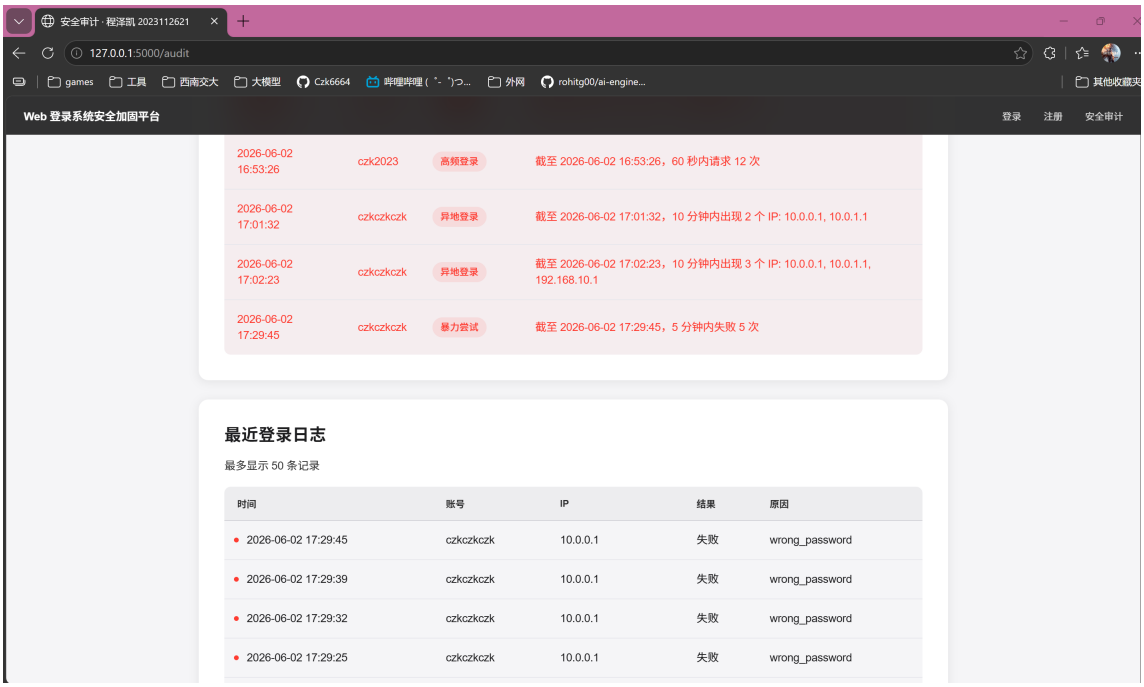


图 **11**　本人本地安全审计页面中的告警明细

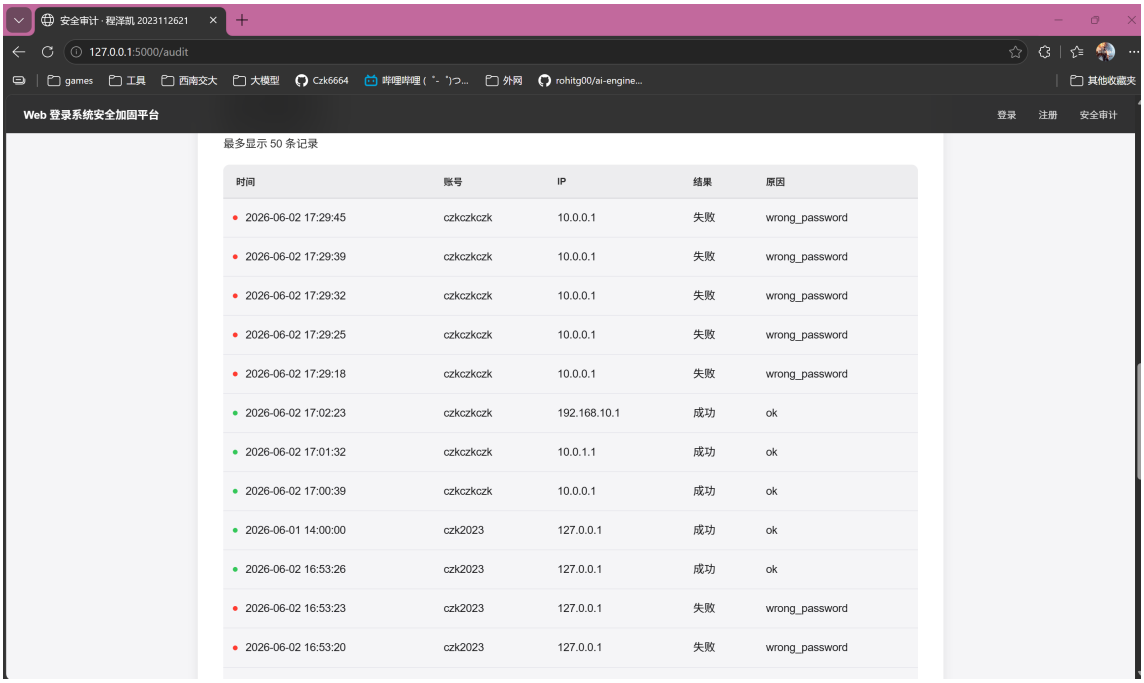


图 **12**　本人本地安全审计页面中的最近登录日志

十一、测试用例

为验证本人设计的安全规则是否真正生效，我编写了自动化测试脚本 `run_tests.py`。每个用例执行前均独立重建数据库以保证隔离，共包含 9 条用例、20 项断言，覆盖正常

功能、安全存储、二次验证模拟、失败限制、异常检测与全量审计，如表 7 所示。

表 **7**　测试用例（≥ 5 条，覆盖不同类型）

编号	测试用例	类型	结果
TC1	注册 + 口令哈希（无明文/pbkdf2 前缀/弱口令被拒）	安全存储	通过
TC2	正确口令登录成功	正常功能	通过
TC3	二次验证模拟码错误被拒，输入 12345 后进入认证流程	二次验证	通过
TC4	连续失败 5 次触发账号锁定	失败限制	通过
TC5	锁定期内正确口令仍被拒绝	失败限制	通过
TC6	异地登录检测（短时多 IP）	异常检测	通过
TC7	非工作时间登录检测	异常检测	通过
TC8	高频登录检测	异常检测	通过
TC9	全量审计汇总（覆盖四类异常）	审计	通过

十二、 运行结果

本人于本机重新运行 `python run_tests.py`，自动化测试最终输出 **20/20 项断言通过**。其中，新增的二次验证用例验证了错误验证码会被拒绝，输入本人设置的固定模拟码 12345 后才进入后续登录认证；全量审计用例生成 23 条模拟登录日志，识别出 10 条告警，四类异常均被覆盖。关键终端输出如下：

```
TC9 Full audit summary
  [PASS] alerts generated: 10
  [PASS] brute force covered
  [PASS] multi-IP login covered
  [PASS] off-hours login covered
  [PASS] high-frequency login covered
  Summary: logs=23, success=6, failed=17, alerts=10

Test result: 20/20 assertions passed
```

此外，本人通过 Flask 测试客户端执行冒烟测试，`/login`、`/register`、`/audit`、`/dashboard` 均返回状态码 200，注册 → 登录 → 个人中心流程完整贯通。结合测试脚本结果，可以确认系统功能、安全规则与审计输出形成了完整闭环。

图 13 为本人使用错误口令连续登录 5 次后的运行截图。系统提示账号锁定 10 分钟，并同步显示“5 分钟内失败 5 次”的暴力尝试告警。

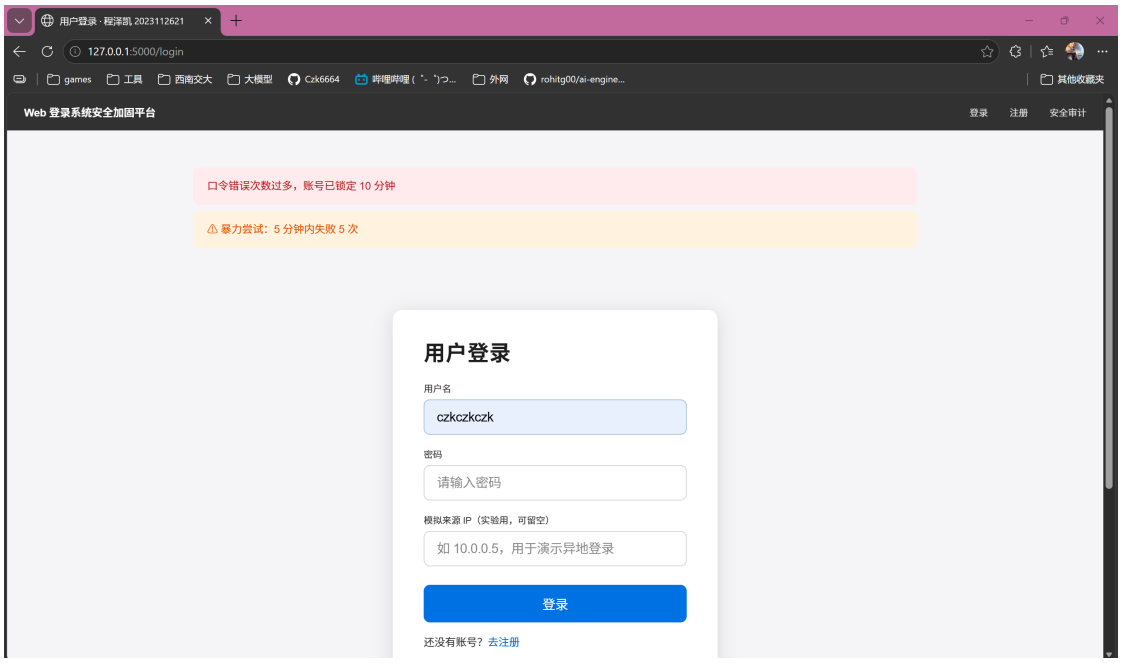


图 **13**　本人连续输错口令后触发账号临时锁定

十三、 问题与改进

13.1 遇到的问题及解决

1. **时间窗口统计**：最初使用简单计数器累加失败次数，无法表达“5 分钟内”的时间语义；改为基于登录日志按时间窗口统计后，时间约束变得精确。
2. **用户枚举风险**：用户不存在与口令错误若返回不同提示，会泄露账号是否存在；统一为“用户名或口令错误”后修复该信息泄露问题，符合 OWASP Top 10 中对敏感信息暴露的防护建议 [3]。
3. **审计可重复性**：重复审计会重复写入告警；通过审计前 `clear_alerts` 加去重集合解决，保证幂等。
4. **二次验证模拟**：为满足任务书中“验证码或二次验证模拟”的要求，本人未接入外部短信或邮箱服务，而是在本地实验环境中设计固定验证码 `12345`，便于答辩现场复现与自动化测试。

13.2 改进方向

- 将当前固定码 `12345` 升级为图形验证码或基于时间的一次性口令（TOTP）二次验证；

- 接入 IP 地理库实现真正的异地判定（当前为同账号多 IP 启发式）；
- 接入限流中间件，对高频来源 IP 自动限流；
- 增加告警推送（邮件 / Webhook）并接入集中式日志分析（如 ELK）。

13.3 答辩可现场验证的参数

修改 `config.py` 中任一阈值后重新运行 `python run_tests.py` 或刷新 `/audit` 即可验证效果，例如：将 `MAX_FAILED_ATTEMPTS` 由 5 改为 3，账号锁定将更早触发；将 `HIGH_FREQ_THRESHOLD` 由 10 改为 5，高频告警数量随之上升。

十四、 总结

本课程设计由本人独立完成，最终实现了一个具备安全加固与审计能力的 Web 登录系统，覆盖任务书要求的全部 7 项必做功能与 4 类异常规则，做到了“有实现、有数据、有流程图、有验证”。系统以 PBKDF2 加盐慢哈希存储口令，加入固定码 `12345` 的二次验证模拟，对失败登录进行基于时间窗口的精确计数并自动锁定账号，记录完整的登录审计日志，并通过实时检测与离线全量审计识别暴力尝试、异地登录、非工作时间登录与高频登录四类异常。

在完成过程中，本人不仅实现了页面与后端功能，还自行设计了集中可调的安全阈值、构造了覆盖正常与异常行为的模拟数据、绘制了五张与代码分支对应的流程图，并编写自动化测试脚本完成验证。最终 20/20 项断言全部通过。通过本次课程设计，本人进一步理解了 Web 认证安全、口令存储、登录审计、异常行为检测以及测试验证之间的关系，也认识到安全系统需要将预防、检测、记录和处置结合起来，而不能只停留在单一功能实现上。

十五、 附录：代码、配置、日志、截图

15.1 项目代码索引

本课程设计代码均由本人在本地完成，主要文件及作用如表 8 所示。

表 **8**　项目代码与材料索引

文件	作用
`config.py`	全局配置与本人设计的安全阈值
`db.py`	SQLite 建表与参数化数据访问
`security.py`	注册、哈希、失败限制、账号锁定与认证
`audit.py`	实时异常检测与离线全量审计
`app.py`	Flask 主程序、路由与页面渲染
`seed_data.py`	本人设计的测试账号与模拟异常日志
`run_tests.py`	9 条自动化测试用例、20 项断言
`docs/截图清单.md`	本人实验截图的获取步骤与放置建议

15.2 关键配置

以下配置集中体现了本人针对课程设计场景设置的安全阈值。答辩时可修改任一参数并重新运行测试。

```
BRUTE_FORCE_WINDOW_MINUTES = 5
MAX_FAILED_ATTEMPTS = 5
ACCOUNT_LOCK_MINUTES = 10
SIMULATED_OTP_CODE = "12345"
MULTI_IP_WINDOW_MINUTES = 10
MULTI_IP_THRESHOLD = 2
OFF_HOURS_START = 23
OFF_HOURS_END = 6
HIGH_FREQ_WINDOW_SECONDS = 60
HIGH_FREQ_THRESHOLD = 10
```

Listing 6: 本人设计的安全阈值（config.py 节选）

参考文献

[1] Paul A. Grassi, James L. Fenton, and Elaine M. Newton. Digital identity guidelines: Authentication and lifecycle management (sp 800-63b). Technical Report SP 800-63B, National Institute of Standards and Technology (NIST), 2020.

[2] Kathleen Moriarty, Burt Kaliski, and Andreas Rusch. Pkcs #5: Password-based cryptography specification version 2.1 (rfc 8018). https://www.rfc-editor.org/rfc/rfc8018, 2017.

[3] OWASP Foundation. Owasp top 10:2021 – the ten most critical web application security risks. https://owasp.org/Top10/, 2021.

[4] OWASP Foundation. Owasp authentication cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html, 2023.